

4/8/26

CO2, AT1

1. Double Linked List :

Initial List - 6 elements

Null $\leftarrow$ [10] $\rightleftarrows$ [20] $\rightleftarrows$ [30] $\rightleftarrows$ [40] $\rightleftarrows$ [50] $\rightleftarrows$ [60] $\rightarrow$
Null

Insert 25 at 3rd position:

Null $\leftarrow$ [10] $\rightleftarrows$ [20] $\rightleftarrows$ [25] $\rightleftarrows$ [30] $\rightleftarrows$ [40] $\rightleftarrows$ [50] $\rightleftarrows$ [60]
 $\rightarrow$ Null

↑
3rd position

Pointer changes:

20. next = 25

25. prev = 20

25. next = 30

30. prev = 25

Delete 2nd position (20)

Null $\leftarrow$ [10] $\rightleftharpoons$ [25] $\rightleftharpoons$ [30] $\rightleftharpoons$ [40] $\rightleftharpoons$ [50] $\rightleftharpoons$ [60] $\rightarrow$ Null

Pointer Changes:

10. next = 25

25. Prev = 10

Analysis:

Insertion and deletion at a given position take $O(n)$ time. A doubly linked list supports traversal in both directions.

2.

Stack operations :

Size = 5

Initial stack:

TOP
↓
[88]
[66]
[33]
[55]
[22]

1. Insert:

TOP
↓
[22]
[55]
[33]
[66]
[88]

2. POP()

TOP
↓
[55]
[33]
[66]
[88]

3. POP()

TOP
↓
[33]
[66]
[88]

4. POP()

TOP
↓
[66]
[88]

5. PUSH(90)

TOP
↓
[90]
[66]
[88]

6. PUSH (36)

TOP
↓
[36]
[90]
[66]
[88]

7. PUSH (11)

TOP
↓
[11]
[36]
[90]
[66]
[88]

8) PUSH (88)

STACK OVERFLOW

(Stack is already full.)

9. POP()

TOP
↓
[36]
[90]
[66]
[88]

10. POP()

TOP
↓
[90]
[66]
[88]

Final Stack

TOP
↓
[90]
[66]
[88]